

DAX Basics

1. What Is DAX?

- DAX (Data Analysis Expressions) is the formula language of Power BI. It is used to create custom calculations on top of your data model
- Every KPI, ratio, running total, and time-based comparison in a Power BI report is written in DAX.
- If you know Excel formulas, DAX syntax will feel familiar. But DAX has one capability Excel formulas fundamentally lack: it is aware of relationships between tables and responds dynamically to every filter, slicer, and visual interaction in the report.

1.1 The three fundamental concepts of DAX

Every DAX formula you will ever write is built on exactly three pillars. Understanding these three concepts deeply makes every DAX function - simple or advanced - predictable.

Pillar	One-line definition	Why it matters
Syntax	Rules for writing a valid DAX formula - name, operator, arguments, parentheses	A formula with wrong syntax throws an error before it even runs. Syntax is the grammar of DAX.
Functions	Built-in operations that transform, aggregate, filter, or compare data	Functions are the vocabulary of DAX. Knowing which function to use - and when - determines what you can build.
Context	The set of filters active when a formula evaluates - determines what data the formula sees	Context is the most important concept in DAX. The same formula produces different results in different contexts. Without understanding context, you cannot debug DAX.

1.2 DAX Syntax - the rules of writing formulas

Every DAX formula follows the same structure regardless of complexity

```
Measure Name = FUNCTION(Table[Column], argument2, argument3)
```

Syntax element	Example	Rule
Measure / Column name	Total Sales	Left of the = sign. No quotes. Spaces allowed.
=(assignment)	=	Required. Separates name from formula.
Function name	SUM	Always uppercase by convention. No spaces.
Table[Column] reference	Orders [Sales]	Table name first, column in square brackets. Both are case-sensitive.
Parenthesis	()	Every function must open and close parentheses. Missing parenthesis = error.
Comma	,	Separates multiple arguments inside a function.
String values	"West"	Text values inside double quotes. Single quotes are for table names with spaces.

Syntax rules to memorise

1. Column references always use Table[Column] format - never just [Column] alone in measures.
2. Table names with spaces use single quotes: 'Order Details'[Sales]

3. Measure references use [MeasureName] - no table prefix.
4. DAX is not case-sensitive for function names but IS case-sensitive for table and column names.

1.3 DAX Functions - the vocabulary

DAX has over 250 built-in functions grouped into categories. Each function takes arguments (inputs), performs an operation, and returns a result. Functions can be nested inside each other - the output of one function becomes the input of another.

```
DIVIDE(SUM(Orders[Profit]), SUM(Orders[Sales]), 0)
```

- SUM() is nested inside DIVIDE() twice
- SUM runs first, DIVIDE receives the two results as its arguments

The major DAX function categories covered in this module and their purpose:

Category	What it does	Functions in this module
Aggregation	Summarise many rows into a single value	SUM, AVERAGE, MIN, MAX, COUNT, COUNTROWS, DISTINCTCOUNT
Math & Statistical	Mathematical operations	DIVIDE, ROUND, ABS
Logical	Conditional evaluation	IF, SWITCH, AND, OR
Filter	Modify or inspect the filter context	CALCULATE, FILTER, ALL, ALLSELECTED, RELATED
Iterator	Row-by-row evaluation over a table	SUMX, AVERAGEX, COUNTX (Module 6+)
Time Intelligence	Date-based calculations	TOTALYTD, SAMEPERIODLASTYEAR (Module 7)

1.4 Context - the most important concept

Context determines what data a DAX formula sees when it evaluates. There are two types of context in DAX - row context and filter context. Every formula operates in one or both of these contexts. Misunderstanding context is the root cause of almost every DAX bug.

The core principle of context

- In DAX, the formula never changes. The context changes.
- The same Total Sales measure returns \$2.3M with no filter, \$725K when filtered to West, and \$234K when filtered to West + Technology.
- One formula. Three contexts. Three results.

2. Calculated Column vs Measure - The Most Important Decision in DAX

- Every time you create a DAX calculation, you must first decide: is this a calculated column or a measure?
- Using the wrong one produces incorrect results silently - no error message, just wrong numbers in your report.

2.1 Complete comparison - all differences explained

Property	Calculated Column	Measure
Where it lives	Stored as a physical column inside a table. Visible in Table View. Appears in the Fields pane with a column icon(it looks like a table with (Σ) at right bottom corner)	Lives in the Fields pane only, with a calculator icon. Has no physical location in any table row . It is represented by calculator icon
When it calculates	Once - at data load or model refresh. The result is stored permanently until	Every single time a visual renders. Any slicer click, filter

	the next refresh.	change, or page navigation triggers recalculation.
Context it uses	Row context - DAX evaluates the formula for one row at a time, seeing only the column values on that specific row.	Filter context - DAX evaluates the formula across all rows that survive the currently active filters in the report.
Slicer / filter aware?	No. A calculated column is computed at load time - it has no knowledge of slicers, page filters, or visual-level filters applied after load.	Yes - fully. Every filter, slicer, matrix row, chart axis, and CALCULATE filter modifies the filter context the measure evaluates in.
Memory storage	Stored in the model's in-memory column store. Every row has a stored value. A 10M-row table with 5 calculated columns = 50M stored values.	Zero memory at rest. The formula definition is stored as text. The result is computed on demand and discarded after rendering.
Can be used as slicer / axis?	Yes - because it has one value per row, Power BI can group and filter by it just like any source column.	No - a measure has no row-level value. It cannot be placed on a slicer, axis, or legend. It can only appear in the Values field.
Aggregation in visuals	When dragged into a visual, Power BI defaults to SUM. This is often wrong - summing row-level profit margins (20% + 15% + 10% = 45%) is mathematically meaningless.	The measure defines its own aggregation logic explicitly. The result is always what the formula specifies, regardless of which visual uses it.
Can reference other measures?	No. A calculated column cannot call a measure - measures require filter context which does not exist during column calculation.	Yes. Measures can reference other measures freely: [Profit Margin %] = DIVIDE([Total Profit], [Total Sales], 0)
Use for	Row-level labels, category flags, classification columns, fixed attributes that do not change based on report filters.	All aggregations, KPIs, ratios, percentages, running totals, and any number that must respond dynamically to user interaction.

Decision rule - one question to ask

- Ask: Does this value need to change when a filter is applied?
- If yes - use a measure.
- If the value is a fixed property of each row (like a category, a flag, or a label) - use a calculated column.

The most common mistake

- Using a calculated column to compute a value that is later shown in a visual.
- When a column with row-level ratios or percentages is dragged into a visual, Power BI SUMs all those row-level values - producing a mathematically wrong answer.
- Always use a measure for any value displayed in a visual.

3. Row Context vs Filter Context

These two concepts are the foundation of all DAX understanding. Every bug in a DAX formula is ultimately a context bug - the formula is evaluating in the wrong context, or the wrong data is visible to it.

3.1 Row context - how calculated columns see data

- Row context is active when DAX evaluates a calculated column.
- DAX processes each row independently - reading the values of that row's columns and writing a result back to that row.

What row context knows:

- The value of every column on the current row
- Nothing about any other row
- Nothing about slicers, page filters, or report-level filters
- Nothing about aggregations across the full column

Visual example - row context in action:

Row	Order ID	Profit	Sales	Calculated Column: Profit Margin %
Row 1	CA-001	\$100	\$500	DIVIDE(100, 500, 0) = 0.20 (20%)
Row 2	CA-002	-\$80	\$400	DIVIDE(-80, 400, 0) = -0.20 (-20%)
Row 3	CA-003	\$60	\$200	DIVIDE(60, 200, 0) = 0.30 (30%)
Row 4	CA-004	\$0	\$0	DIVIDE(0, 0, 0) = 0 (alternate result)

This how DAX evaluates:

- Row 1 using only Row 1's Profit and Sales.
- It evaluates Row 2 using only Row 2's values.
- Each row is completely independent.
- The region filter on the report, the year slicer, the page filter - none of these affect what the calculated column writes into each row.

Critical rule about aggregations in row context

- If you write SUM(Orders[Sales]) inside a calculated column, DAX ignores the row context entirely.
- SUM is an aggregation function - it scans the entire column unconditionally and returns \$2,297,201 (the grand total) on every single row.
- This is not a bug - aggregation functions were designed for filter context, not row context.

3.2 Filter context - how measures see data

- Filter context is active when DAX evaluates a measure.
- The filter context defines exactly which rows of each table are visible to the measure.
- Everything outside the filter context is invisible - it does not exist as far as the measure is concerned.

Sources of filter context in a Power BI report:

- Slicer selections - a Region slicer set to West filters the Orders table to West rows only
- Visual axis values - a bar chart with Category on the axis creates one filter context per bar
- Matrix rows and columns - the cell at West row / Technology column has two simultaneous filters
- Page-level and report-level filters in the Filters pane
- CALCULATE - programmatically adds, removes, or overrides filters (Module 6)

Visual example - filter context in action:

Active filter context	Rows visible to the measure	Total Sales result
No filter (card visual)	All 9,994 rows	\$2,297,201
Region slicer = West	Only West rows (~3,156 rows)	\$725,458
Region = West AND Category = Technology	West + Technology rows only (~580 rows)	\$152,914
Category = Furniture (from bar chart axis)	Only Furniture rows (~2,121 rows)	\$741,999

The measure formula Total Sales = SUM(Orders[Sales]) is identical in every scenario above. Only the filter context changes. This is the defining behaviour of measures - one formula, unlimited contexts, always the correct answer for each context.

3.3 Comparison - row context vs filter context

Property	Row Context	Filter Context
Active in	Calculated columns	Measures
What DAX sees	One row - all columns of that row only	All rows surviving the active filters

Slicer aware?	No	Yes - fully
Changes when user interacts?	No - fixed at data load	Yes - recalculates on every interaction
Column references (e.g. [Sales])	Value on the current row	Not valid without an aggregation function wrapper
Aggregation functions (e.g. SUM)	Scan the entire column, ignoring the row	Aggregate only the visible (filtered) rows

3.4 Filter direction - how context flows through the model

Filters always travel from the one side (dimension table) to the many side (fact table) through relationships. Understanding this direction is critical for writing multi-table DAX measures.

Relationship	Direction	Effect on the measure
People → Orders (via Region)	People → Orders	Selecting West in a slicer filters People[Region] = West, which propagates to Orders - only West orders are visible to the measure
Product → Orders (via Product ID)	Product → Orders	Selecting Technology filters Product[Category] = Technology, propagating to Orders - only Technology orders are visible
Date → Orders (via Order Date)	Date → Orders	Selecting Year = 2017 filters Date[Year] = 2017, propagating to Orders - only 2017 orders are visible

Filter direction is one-way by default

- Filters flow FROM dimension TO fact table - never from fact to dimension.
- Selecting a value in the Orders table does not automatically filter the People or Product tables.
- Bidirectional filtering exists but introduces complexity and should be avoided unless specifically required.

4. DAX Functions - Complete Reference for Module 5

Every function below follows the same documentation format: Syntax (Courier New), Arguments (each explained individually), and a concrete Example from the Superstore dataset. Study the syntax format carefully - interviews often ask you to write functions from memory.

How to read function syntax

- Angle brackets < > denote a required argument.
- Square brackets [] denote an optional argument.
- A comma separates each argument.
- The function name is always in UPPERCASE by convention.

4.1 Aggregation Functions

Aggregation functions take a column or table as input and return a single summarised value. They always operate in filter context - they aggregate only the rows currently visible to the measure.

SUM

Type: Aggregation

Syntax

```
SUM( <ColumnName> )
```

Arguments

- **<ColumnName>** - The column containing numeric values to add. Must reference a single column using Table[Column] format. Non-numeric values are ignored.

Example

```
Total Sales = SUM(Orders[Sales])
```

- Adds all Sales values in the Orders table for rows visible in the current filter context.
- If Region slicer = West, only West sales are summed. Result on the full dataset: \$2,297,201.

AVERAGE

Type: Aggregation

Syntax

```
AVERAGE ( <ColumnName> )
```

Arguments

- **<ColumnName>** - The column containing numeric values to average. Blank values are excluded from both the numerator and the denominator - they do not count as zero.

Example

```
Avg Order Value = AVERAGE(Orders[Sales])
```

- Returns the arithmetic mean of the Sales column for all visible rows.
- Blank Sales cells are excluded entirely.
- Equivalent to $\text{SUM(Orders[Sales])} / \text{COUNTROWS(Orders)}$ - but AVERAGE handles blanks correctly while the manual formula does not.

MIN

Type: Aggregation

Syntax

```
MIN ( <ColumnName> )
```

Arguments

- **<ColumnName>** - The column to find the minimum value in. Works on numeric, date, and text columns. For text, returns the first value alphabetically.

Example

```
Earliest Order Date = MIN(Orders[Order Date])
```

Returns the smallest value in the column for the current filter context. For a date column, this gives the earliest date. For Sales, it gives the single lowest individual sale value.

MAX

Type: Aggregation

Syntax

```
MAX( <ColumnName> )
```

Arguments

- **<ColumnName>** - The column to find the maximum value in. Works on numeric, date, and text columns. For text, returns the last value alphabetically.

Example

```
Latest Order Date = MAX(Orders[Order Date])
```

- Returns the largest value in the column for the current filter context.
- MAX(Orders[Order Date]) is used extensively in time intelligence formulas to identify the 'current' date being evaluated in a visual.

COUNT

Type: Aggregation

Syntax

```
COUNT( <ColumnName> )
```

Arguments

- **<ColumnName>** - The column whose non-blank values are counted. Blank cells are skipped - they are NOT counted as zero. This makes COUNT unreliable for row counting if any key column has blanks.

Example

```
Order Lines = COUNT(Orders[Order ID])
```

- Counts non-blank values in the Order ID column. Returns 9,994 on the full Superstore dataset.
- However, if any Order ID were blank, this count would silently be wrong. Use COUNTROWS instead for reliable row counts.

COUNTROWS

Type: Aggregation

Syntax

```
COUNTROWS ( <Table> )
```

Arguments

- **<Table>** - The table whose rows are counted unconditionally. Unlike COUNT, COUNTROWS does not reference a column - it counts every row in the table regardless of any column's value.

Example

```
Total Rows = COUNTROWS(Orders)
```

- Counts all rows in the Orders table visible in the current filter context.
- Returns 9,994 with no filter. Returns ~3,156 when Region = West.
- This is always the preferred function for row counting - it never silently skips blank values.

DISTINCTCOUNT

Type: Aggregation

Syntax

```
DISTINCTCOUNT ( <ColumnName> )
```

Arguments

- **<ColumnName>** - The column to count unique values in. Each unique value is counted exactly once, regardless of how many times it appears. Blank values count as one distinct value if present.

Example

```
Total Orders = DISTINCTCOUNT (Orders[Order ID])
```

Counts unique Order IDs. Returns 5,009 on the full Superstore dataset - significantly fewer than 9,994 total rows because multi-product orders generate multiple rows sharing one Order ID. For business reporting, this is the correct measure for 'number of orders placed'.

SUMX

Type: Iterator (row-by-row aggregation)

Syntax

```
SUMX ( <Table>, <Expression> )
```

Arguments

- **<Table>** - The table to iterate over - DAX processes every row of this table one at a time.
- **<Expression>** - A formula evaluated for each row during iteration. The result of each row evaluation is collected, then summed at the end.

Example

```
Total Revenue =  
SUMX (  
    Orders,
```

```
Orders[Quantity] * Orders[Sales]
)
```

- For each row in Orders, multiplies Quantity by Sales to get row-level revenue, then sums all those row-level results.
- This is different from SUM - SUMX can evaluate complex row-level expressions before aggregating, which SUM cannot.

4.2 Math and Statistical Functions

DIVIDE

Type: Math

Syntax

```
DIVIDE( <Numerator>, <Denominator> [,<AlternateResult>] )
```

Arguments

- **<Numerator>** - The value to be divided - the top number of the fraction.
- **<Denominator>** - The value to divide by - the bottom number. If this evaluates to zero or blank, the AlternateResult is returned instead of an error.
- **[<AlternateResult>]** - Optional. The value returned when the denominator is zero or blank. Standard practice is 0. If omitted, BLANK() is returned on zero division.

Example

```
Profit Margin % =
DIVIDE(
    SUM(Orders[Profit]),
    SUM(Orders[Sales]),
    0
)
```

Working:

- Divides total profit by total sales for the current filter context.
- Returns 0 if Sales = 0, keeping the report visually clean.

- The / operator would return an error for the same scenario - breaking that cell in the visual. DIVIDE is mandatory in production reports.

ROUND

Type: Math

Syntax

```
ROUND( <Number>, <NumDigits> )
```

Arguments

- **<Number>** - The value or expression to round. Can be a direct number, a column reference, or another DAX expression.
- **<NumDigits>** - The number of decimal places to round to. 0 = whole number. 2 = two decimal places. Negative values round to tens, hundreds, etc.

Example

```
Rounded Sales = ROUND(SUM(Orders[Sales]), 0)
```

Working:

- Returns the sum of Sales rounded to the nearest whole number.
- Useful for presenting clean totals in visuals without decimal noise.

ABS

Type: Math

Syntax

```
ABS( <Number> )
```

Arguments

- **<Number>** - The value or expression to return the absolute (non-negative) value of. Negative numbers become positive. Positive numbers are unchanged.

Example

```
Absolute Profit = ABS(SUM(Orders[Profit]))
```

Working:

- Returns the total profit as a positive number regardless of sign.

- Useful for loss analysis visuals where you want to display the magnitude of loss without negative signs.

4.3 Logical Functions

IF

Type: Logical

Syntax

```
IF( <LogicalTest>, <ResultIfTrue> [, <ResultIfFalse>] )
```

Arguments

- **<LogicalTest>** - A condition that evaluates to TRUE or FALSE. Can use comparison operators: =, <>, >, <, >=, <=
- **<ResultIfTrue>** - The value returned when the LogicalTest is TRUE. Can be a number, text, or another DAX expression.
- **[<ResultIfFalse>]** - Optional. The value returned when the LogicalTest is FALSE. If omitted, returns BLANK() when FALSE.

Example

```
Sales Band =  
IF(Orders[Sales] > 1000, "High",  
    IF(Orders[Sales] > 500, "Medium", "Low")  
)
```

Working:

- checks each row's Sales value and assigns a text label.
- High for Sales > 1000, Medium for Sales 500-1000, Low for everything else. Nested IF handles three tiers. This column can then be used as a slicer or visual axis.

SWITCH

Type: Logical

Syntax

```
SWITCH( <Expression>, <Value1>, <Result1> [, <Value2>, <Result2>]... [, <Else>] )
```

Arguments

- **<Expression>** - The value or expression to test against each Value. When used with SWITCH(TRUE(), ...), each Value becomes a logical condition instead of a direct match.
- **<Value1>, <Result1>** - A pair: if Expression matches Value1, return Result1. Multiple pairs can be chained.
- **[<Else>]** - Optional. The value returned when no Value matches. Equivalent to the final ELSE in a nested IF.

Example

```
Profit Category =  
SWITCH(  
    TRUE() ,  
    SUM(Orders[Profit]) > 50000, "High Profit",  
    SUM(Orders[Profit]) > 10000, "Medium Profit",  
    SUM(Orders[Profit]) > 0,      "Low Profit",  
    "Loss"  
)
```

- SWITCH(TRUE(), ...) is the professional pattern for multi-condition branching.
- Each condition is evaluated in order - the first TRUE condition wins.
- Cleaner than deeply nested IF statements.
- The final string 'Loss' is the Else - returned when Profit is zero or negative.

AND / OR

Type: Logical

Syntax

```
AND( <Logical1>, <Logical2> )  
OR( <Logical1>, <Logical2> )
```

Arguments

- **<Logical1>** - First condition - evaluates to TRUE or FALSE.
- **<Logical2>** - Second condition - evaluates to TRUE or FALSE. AND returns TRUE only when both are TRUE. OR returns TRUE when at least one is TRUE.

Example

```
Using && and || operators (preferred shorthand)  
High Value Loss =  
CALCULATE (  
    COUNTROWS (Orders) ,  
    Orders[Sales] > 500 && Orders[Profit] < 0  
)
```

- In DAX, && is the shorthand for AND, and || is the shorthand for OR.
- Both are preferred over the AND() and OR() functions in practice because they are more readable inside CALCULATE and FILTER expressions.

4.4 Filter Functions

CALCULATE

Type: Filter - most important DAX function

Syntax

```
CALCULATE( <Expression> [, <Filter1>] [, <Filter2>] ... )
```

Arguments

- **<Expression>** - The measure or expression to evaluate. This is what you are calculating.
- **[<Filter1>], [<Filter2>]** - Optional filter arguments. Each one modifies the filter context before the expression evaluates. Multiple filters are AND conditions - all must be satisfied. Covered in full detail in Module 6.

Example

```
West Sales =  
CALCULATE (  
    SUM(Orders[Sales]) ,  
    People[Region] = "West"  
)
```

- Evaluates SUM(Orders[Sales]) inside a modified filter context where Region is forced to West - regardless of what the report's slicers say.
- This is an introduction to CALCULATE. The full depth of filter context manipulation - including ALL, ALLSELECTED, REMOVEFILTERS, and context transition - is the subject of Module 6.

FILTER

Type: Filter

Syntax

```
FILTER( <Table>, <FilterExpression> )
```

Arguments

- **<Table>** - The table to iterate over. FILTER processes every row of this table and returns only the rows where FilterExpression is TRUE.
- **<FilterExpression>** - A condition evaluated for each row. Rows where the condition is TRUE are kept. Rows where it is FALSE are excluded from the returned table.

Example

```
Discounted Loss Orders =  
CALCULATE (  
    COUNTROWS (Orders) ,  
    FILTER (  
        Orders ,  
        Orders[Discount] > 0.2 && Orders[Profit] < 0  
    )  
)
```

- FILTER iterates through every row of Orders and keeps only rows where Discount > 20% AND Profit < 0 simultaneously on the same row.
- This row-by-row evaluation is what makes FILTER necessary when two conditions must both be true on the same row - which simple column filter arguments inside CALCULATE cannot guarantee.

ALL

Type: Filter

Syntax

```
ALL ( <TableOrColumn> )
```

Arguments

- **<TableOrColumn>** - Either a table name (removes all filters from the entire table) or a Table[Column] reference (removes filters from that specific column only). ALL returns the full unfiltered table or column.

Example

```
% of Total Sales =  
DIVIDE (  
    SUM(Orders[Sales]) ,  
    CALCULATE (SUM(Orders[Sales]) , ALL(Orders)) ,  
    0  
)
```

Working:

- ALL(Orders) removes every filter from the Orders table - returning the grand total sales as the denominator regardless of what Region or Category is selected.

- The numerator SUM(Orders[Sales]) still uses the current filter context.
- Result: each category bar shows its percentage contribution to the grand total.

RELATED

Type: Filter / Relationship

Syntax

```
RELATED ( <ColumnName> )
```

Arguments

- **<ColumnName>** - A column from a related table - must be on the one side of a relationship. RELATED traverses the relationship from the current row's table (many side) to the related table (one side) and returns the matching value.

Example

```
Sales Person = RELATED (People[Person])
```

- Used in a calculated column in the Orders table.
- For each order row, RELATED traverses the Orders→People relationship using the Region value, and returns the Person assigned to that region.
- This is the DAX equivalent of VLOOKUP - but it uses the data model relationship instead of searching a column.

5. Creating Measures and Calculated Columns in Power BI

5.1 Creating a measure

1. In the Fields pane, click the _Measures table (create one if it does not exist - see Section 7)
2. Right-click → New Measure
3. The formula bar opens at the top
4. Type: Measure Name = DAX formula

5. Press Enter

5.2 Creating a calculated column

6. In the Fields pane, click the target table (e.g. Orders)
7. Right-click → New Column
8. Type: Column Name = DAX formula
9. Press Enter - the column appears in Table View with a value for every row

5.3 Where to store measures - best practice

1. Never store measures inside fact or dimension tables.
2. Create a dedicated _Measures table (Home → Enter Data → name it _Measures → Load → delete the default column in Power Query).
3. The underscore prefix sorts it above all other tables in the Fields pane.
4. A table containing only measures displays a calculator icon - not a table icon.

6. Mistakes Made and Lessons Learned

Every mistake below was made during Module 5 practice. Each one demonstrates a precise conceptual gap - not just a wrong answer.

Mistake 1: Used ALLSELECTED as a fix for grand total in calculated column (three times)

What happened: Attempted to use **ALLSELECTED** to fix a grand total issue in a calculated column. The problem was **SUM** ignoring row context.

Why it was wrong: **ALLSELECTED** removes filters and is the opposite of adding a specific filter. It does not fix context transition problems.

Fix: Use **CALCULATE** with **FILTER** and **EARLIER** for context transition.

Lesson: `ALLSELECTED` only removes slicer filters - never use it as a general fix.

Mistake 2: Used calculated column for blended category margin

What happened: Used a calculated column for a row-level margin.

Why it was wrong: Dragging a row-level margin column into a visual causes Power BI to `SUM` all margins (e.g., 20% + 15% + 10% = 45%), which is mathematically meaningless.

Fix: Use `DIVIDE(SUM([Profit]), SUM([Sales]), 0)` as a measure.

Lesson: The measure computes the blended margin for the visible rows, which is the correct business answer.

Mistake 3: Used `COUNT(column)` instead of `COUNTROWS` for row counts

What happened: Used `COUNT(column)` for row counting.

Why it was wrong: `COUNT` skips blank values silently. If any `Order ID` were blank, the count would be wrong with no error or warning.

Fix: Always use `COUNTROWS(Orders)` for row counts.

Lesson: `COUNTROWS` counts every row unconditionally regardless of column values.

Mistake 4: Referenced a calculated column inside a measure

What happened: Referenced a calculated column, like `Orders[Discount Flag]`, inside a measure.

Why it was wrong: This creates an implicit dependency. If the column is renamed or deleted, the measure breaks silently.

Fix: Filter the source column directly inside `CALCULATE`: `Orders[Discount] > 0.2`.

Lesson: Never reference a calculated column from a measure.

Mistake 5: Used `/` operator instead of `DIVIDE`

What happened: Used the `/` operator for division.

Why it was wrong: The / operator returns an error when the denominator is zero, propagating visually as a blank or error symbol.

Fix: Always use `DIVIDE (numerator, denominator, 0)` in production.

Lesson: The third argument prevents divide-by-zero errors from breaking the visual.

Mistake 6: Wrote measure formula without SUM wrappers

What happened: Wrote a measure formula like `Orders[Profit] / Orders[Sales]` without aggregation wrappers.

Why it was wrong: Column references without aggregation functions are invalid in measures, resulting in an error.

Fix: In a measure, always wrap column references in aggregation functions:

`DIVIDE (SUM(Orders[Profit]), SUM(Orders[Sales]), 0).`

Lesson: Always wrap column references in aggregation functions in a measure.

Mistake 7: Described filter direction as Orders → Product (reversed)

What happened: Described filter direction as Orders → Product (from fact to dimension).

Why it was wrong: Filters flow FROM dimension (one side) TO fact table (many side). Orders is the fact table and does not push filters to Product.

Fix: Always state: dimension filters fact.

Lesson: Filters flow: People → Orders. Product → Orders. Date → Orders. Never the reverse.

The ALLSELECTED pattern - never use as a default fix

- ALLSELECTED is the most misused function in beginner DAX.
- It has one specific purpose: remove slicer filters while keeping visual-level filters - used in subtotal percentage calculations.

- It is not a fix for row context problems, grand total issues, or any context transition scenario. If you find yourself reaching for ALLSELECTED to fix an unexpected result, stop - the real issue is almost always a context misunderstanding that requires CALCULATE + FILTER + EARLIER.

7. DAX Quick Reference Card

Function	Syntax	Purpose
SUM	SUM (Table[Column])	Total of all values in a column for the filter context
AVERAGE	AVERAGE (Table[Column])	Mean of all non-blank values in a column
MIN	MIN (Table[Column])	Smallest value in a column for the filter context
MAX	MAX (Table[Column])	Largest value in a column for the filter context
COUNT	COUNT (Table[Column])	Count of non-blank values. Use COUNTROWS instead.
COUNTROWS	COUNTROWS (Table)	Count of all rows unconditionally. Preferred for row counts.
DISTINCTCOUNT	DISTINCTCOUNT (Table[Column])	Count of unique values in a column
SUMX	SUMX (Table, Expression)	Row-by-row evaluation then sum of results
DIVIDE	DIVIDE (Num, Den, Alt)	Safe division - returns Alt when Den = 0
ROUND	ROUND (Number, NumDigits)	Round to specified decimal places
ABS	ABS (Number)	Absolute (non-negative) value
IF	IF (Test, IfTrue, IfFalse)	Conditional - returns one of two values based on a condition
SWITCH	SWITCH (TRUE(), cond, result, ...)	Multi-condition branching - cleaner than nested IF
CALCULATE	CALCULATE (Expr, Filter1, ...)	Evaluate expression in modified filter context - Module 6
FILTER	FILTER (Table, Condition)	Returns rows where condition is TRUE - row-by-row

ALL	ALL (Table or Column)	Removes all filters from table or column
RELATED	RELATED (Table[Column 1])	Fetch value from related table - many to one direction